Ingeniería de soluciones con AI

Semestre 2026-1

Docente:

Francisco Macaya

Noticias: *Yann LeCun, pionero en AI, reclama que los LLMs no son capaces de entender el mundo físico.*

WIRED SECURITY POLITICS THE BIG STORY BUSINESS SCIENCE CULTURE REVIEWS NEWSLETTERS SUBSCRIBE

MAXWELL ZEFF BUSINESS MAR 10, 2026 1:00 AM

Yann LeCun Raises $1 Billion to Build AI That Understands the Physical World

Meta's former chief AI scientist has long argued that human-level AI will come from mastering the physical world, not language. His new startup, AMI, aims to prove it.

Nota: https://www.wired.com/story/yann-lecun-raises-dollar1-billion-to-build-ai-that-understands-the-physical-world/

Clase 1.2: *Técnicas de Prompt Engineering*

Se cubrirán los siguientes tópicos en la clase:

- Zero-shot.
- Few-shot learning.
- Chain-of-thought.
- Prompts Especializados.

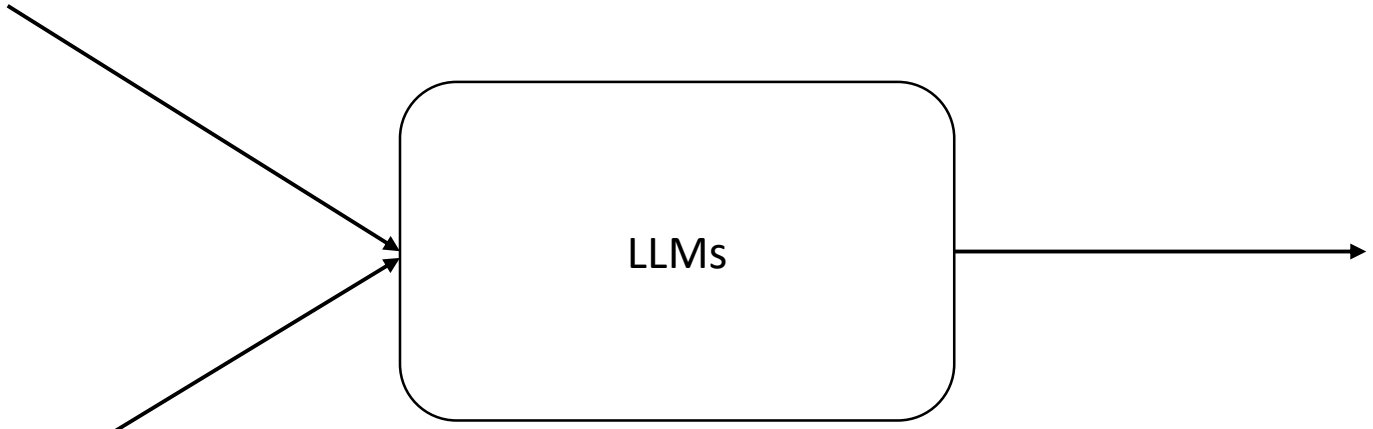
No obstante, esta clase ***no cubrirá*** los siguientes tópicos:

- fine-tunning.
- RAG.

Reminder: *Que hemos visto hasta ahora…*

Los LLMs tiene una estructura de mensajes:
-Prompt de Sistema
-Human Message
-Asistent Message

Los LLMs tiene una parámetros de entrada
-Tempuratura
-Max token



Los LLMs entregan los siguiente output:
- Texto generado (response/completion)
- Usage_prompt (Prompt, completion and total tokens)
- Finished reason.

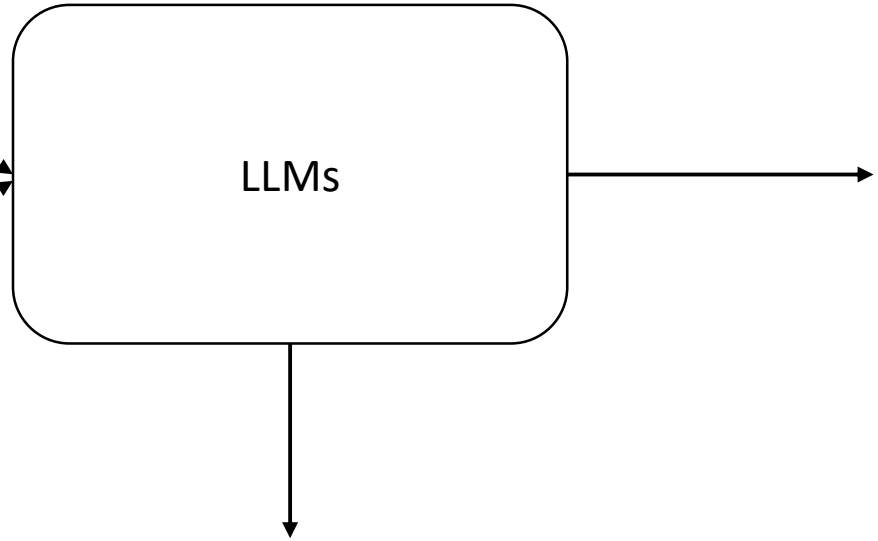
Nota: https://arxiv.org/pdf/2509.04664

Reminder: *Que hemos visto hasta ahora…*

Los LLMs tienen una estructura de mensajes, compuesta por:
-Prompt de sistema
-Human message
-Assistant message

Prompt Engineering: Estructurar la llamada.

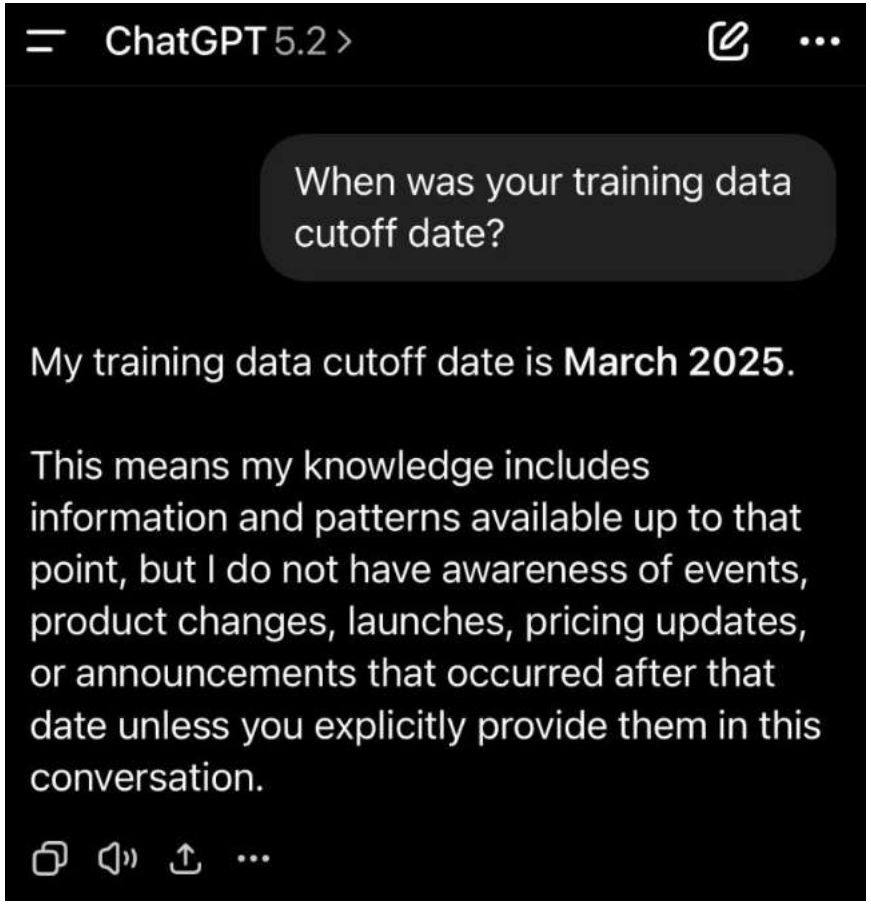
Los LLMs tienen parámetros de entrada, entre los que se encuentran:
- Temperatura
- Max tokens



Los proveedores más conocidos de LLMs son **Meta, OpenAI, Anthropic, Google**

Los LLMs entregan los siguientes outputs:
1-) Texto generado (response/completion)
2)- Usage prompt: incluye prompt tokens, completion tokens y total tokens.
- Finish reason: indica la razón por la cual el modelo terminó de generar la respuesta.

Why: *¿Por qué los LLMs no me responden como quiero?*



Los modelos de lenguaje son entrenados con toda la información del internet, y tiene una fecha de corte de entrenamiento (“cut-off”).

- Los LLMs son generalistas.
- Tienen una fecha de corte.
- Se construyen en base a datos públicos, no privados.

Hay técnicas para resolver estos ***problemas.***

***Hallucinate**: El talón de Aquiles de los LLMs.*

Why Language Models Hallucinate

Adam Tauman Kalai* OpenAI
Ofir Nachum OpenAI
Santosh S. Vempala† Georgia Tech
Edwin Zhang OpenAI

September 4, 2025

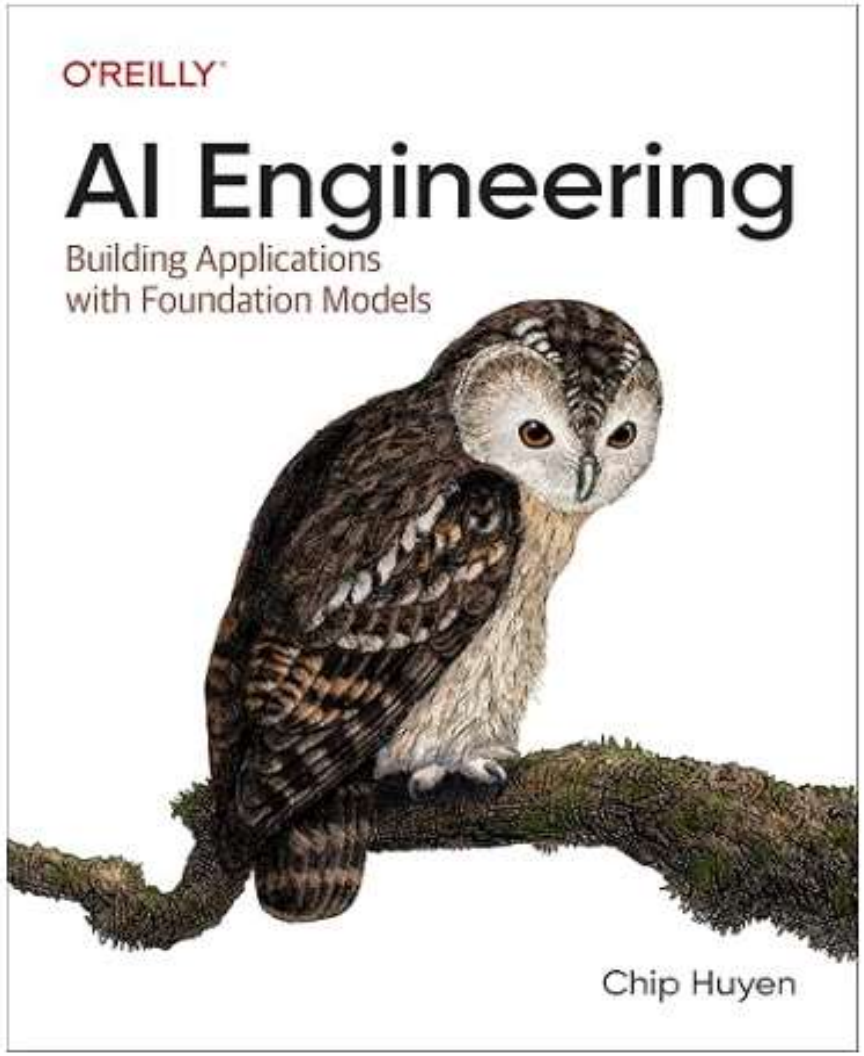
{v1 [cs.CL] 4 Sep 2025

Abstract

Like students facing hard exam questions, large language models sometimes guess when uncertain, producing plausible yet incorrect statements instead of admitting uncertainty. Such "hallucinations" persist even in state-of-the-art systems and undermine trust. We argue that language models hallucinate because the training and evaluation procedures reward guessing over acknowledging uncertainty, and we analyze the statistical causes of hallucinations in the modern training pipeline. Hallucinations need not be mysterious—they originate simply as errors in binary classification. If incorrect statements cannot be distinguished from facts, then hallucinations in pretrained language models will arise through natural statistical pressures. We then argue that hallucinations persist due to the way most evaluations are graded—language models are optimized to be good test-takers, and guessing when uncertain improves test performance. This "epidemic" of penalizing uncertain responses can only be addressed through a socio-technical mitigation: modifying the scoring of existing benchmarks that are misaligned but dominate leaderboards, rather than introducing additional hallucination evaluations. This change may steer the field toward more trustworthy AI systems.

Nota: https://arxiv.org/pdf/2509.04664

O'REILLY: AI Engineering



"Anyone can communicate, but not everyone can communicate effectively".

Prompt Engineering *es una técnica para mejorar las respuestas de los modelos, buscando consistencia, reducción de alucinaciones y personalización de los LLMs.*

Prompt Engineering *es una técnica para mejorar las respuestas de los modelos, buscando consistencia, reducción de alucinaciones y personalización de los LLMs.*

¿Es la solución perfecta?

Prompt Engineering *es una técnica para mejorar las respuestas de los modelos, buscando consistencia, reducción de alucinaciones y personalización de los LLMs.*

¿Es la solución perfecta?

No, pero ayuda.

Tipos de comunicaciones: Sistema, humano y asistente.

```
messages =
[
    system
    user
    assistant
    user
    assistant
    ...
]
```

→ *Prompt System: d*efine el comportamiento del modelo.

→ *User Message:* mensaje del usuario.

→ *Assistant Message:* respuesta del modelo de lenguaje. Si existe una conversación, cada interacción se debe almacenar para entregarla como parámetro al request.

Las técnicas de ***Prompt Engineering*** buscan personalizar y ajustar el Prompt System, no los demás mensajes.

¿Cuáles son las técnicas de Prompt Engineering?

Zero – Shot Prompting

Mensaje directo al LLM, sin ningún ejemplo previo.

Ventajas:

- Implementación fácil y rápida.

Desventajas:

- Poco confiable.
- Los problemas reales son complejos, entonces muy poco usado en la industria.

```
Instrucción clara + Contexto → Resultado esperado
"Clasifica el siguiente email como spam o no spam: [email]"
```

Nota: https://blog.langchain.com/few-shot-prompting-to-improve-tool-calling-performance/

Few-Shot Prompting

Mensaje directo al LLM, pero con ejemplos de entradas y salidas. Le doy ejemplos al LLM de como quiero que trabaje.

Ventajas:

- Mas preciso en la tarea. Le enseño como comportarse.

Desventajas:

- Output más estándar.
- Necesito tener los ejemplos.
- El prompt puede tomar más tiempo en construirlo.

```
Ejemplos + Patrón + Nueva entrada → Resultado
Ejemplo 1: Input → Output
Ejemplo 2: Input → Output
Nueva entrada: [input] → ?
```

Chain-of-Thought (COF)

Antes de tener una respuesta, lo correcto es "pensar". El COF busca que los LLMs construyan una secuencia de pasos antes de entregar la respuesta, como si estuvieran haciendo el calculo matemático de una expresión compleja en "Voz Alta".

Ventajas:

- Mayor confiabilidad.
- Trazabilidad del pensamiento respecto a la respuesta dada.
- Descompone, razona y construye la respuesta.
- Debugging.

Desventajas:

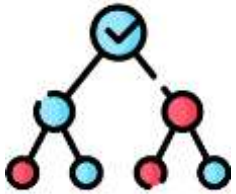
- Toma mayor tiempo de inferencia.
- No aplica para todos los casos.

```
Problema + "Piensa paso a paso" → Razonamiento + Solución
"Resuelve: 23 × 17. Piensa paso a paso."
```

Técnicas avanzadas de Prompting

Tree of Thoughts (ToT)

Tree of Thoughts permite al modelo explorar múltiples caminos de razonamiento de manera sistemática, evaluando y seleccionando las mejores opciones.



Self-Consistency

Self-Consistency mejora la confiabilidad generando múltiples respuestas y seleccionando la más consistente.

Program-Aided Language Models (PAL)

PAL combina el **razonamiento de LLMs con la precisión** de código ejecutable para cálculos exactos. Se utiliza para cálculos numéricos más complejos, que necesitan una secuencia de etapas.

Meta-Prompting

Meta-prompting usa el LLM para generar y optimizar prompts para tareas específicas.

Prompt Chaining

Prompt chaining conecta múltiples prompts en secuencia, donde la salida de uno alimenta al siguiente

¿Cómo construir un buen prompt?

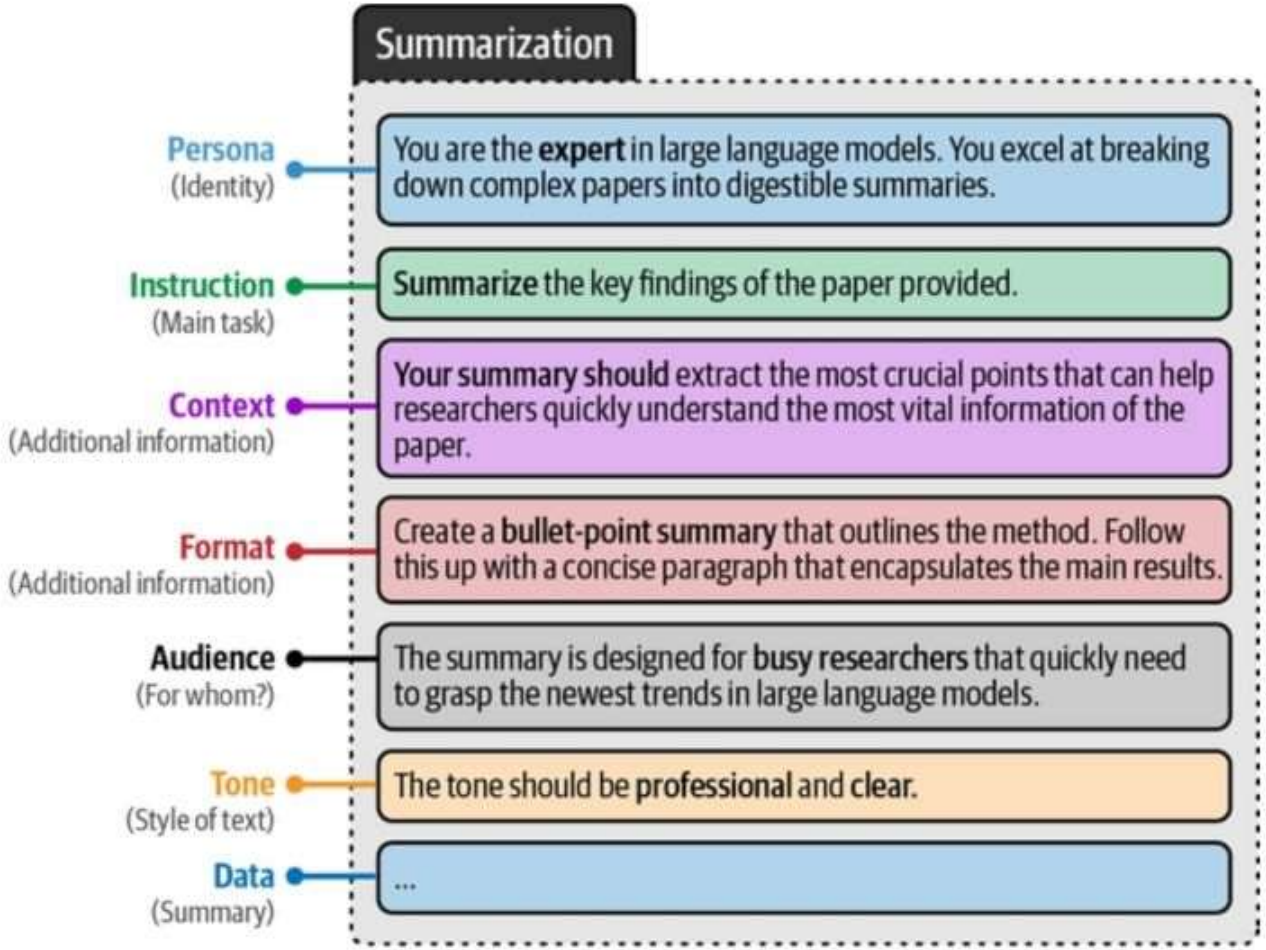
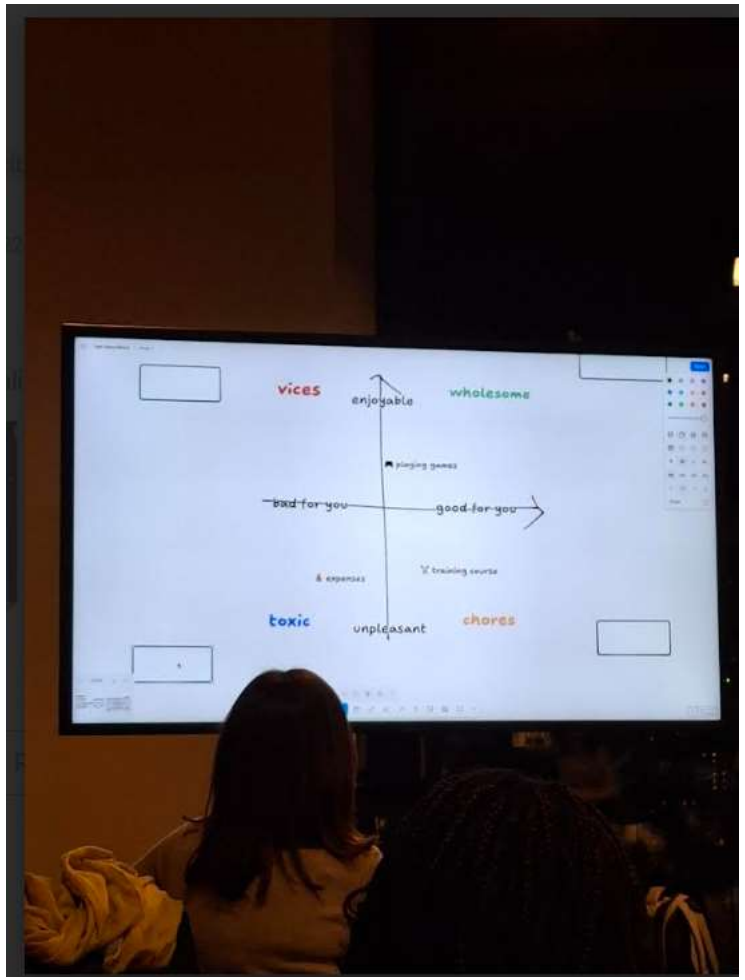


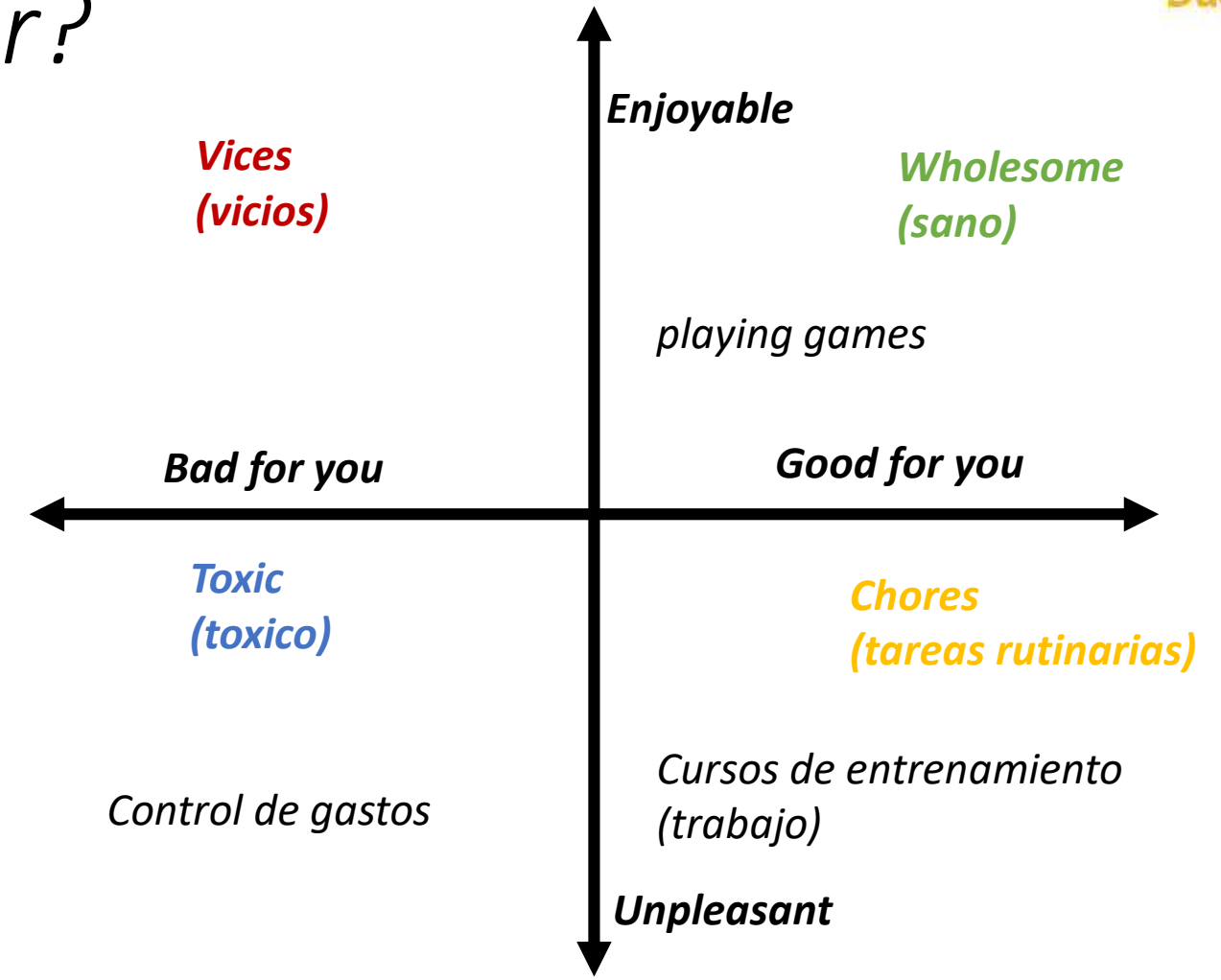
Figure 6-11. An example of a complex prompt with many components.

Simplicity is the ultimate sophistication. (Leonardo da Vinci)

Idea: ¿Qué construir?



Cementerios de Startup: *https://startups.rip/*



- **Wholesome (actividades sanas):** No uses IA. Disfrútalas.
  Toxic (actividades tóxicas): Usa IA para automatizarlas o eliminarlas. No queremos lidiar con ellas.
  Chores (tareas domésticas o rutinarias): Usa IA para hacerlas más placenteras. *(Sigan pensando.)*
  Vices (vicios): Usa IA para evitar hacerlas.